

Explain Fair Pairs Algorithm

Absolutely. This is a very good problem for learning a **general pattern**: **sort + binary search + convert a pair condition into a range condition**.

The code is solving **LeetCode 2563 – Count the Number of Fair Pairs**.

The important thing is not just memorizing `lower_bound / upper_bound`. You want to understand **how we arrive at this solution from the problem statement**.

1. First understand the problem

We have an array:

```
nums = [0, 1, 7, 4, 4, 5]
lower = 3
upper = 6
```

A pair (i, j) is **fair** if:

```
lower <= nums[i] + nums[j] <= upper
```

and:

```
i < j
```

For example:

```
nums = [0, 1, 7, 4, 4, 5]
```

Some pairs:

$0 + 1 = 1$	✗
$0 + 4 = 4$	✓
$0 + 5 = 5$	✓
$1 + 4 = 5$	✓
$1 + 5 = 6$	✓
$7 + 4 = 11$	✗

We need to count all such pairs.

2. Brute-force approach

The most obvious approach is:

```
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {

        if (lower <= nums[i] + nums[j] &&
            nums[i] + nums[j] <= upper) {

            ans++;
        }
    }
}
```

This checks every pair.

Number of pairs:

```
n * (n - 1) / 2
```

So complexity is:

```
O(n2)
```

If n is large, this becomes too slow.

So we need to exploit some property.

3. The key intuition: Fix one number

This is the most important idea in the entire solution.

Instead of thinking:

"How do I find all pairs?"

think:

"What if I fix one element and find all possible elements that can pair with it?"

Suppose:

```
nums[i] = 4
```

We want:

```
lower <= nums[i] + nums[j] <= upper
```

Suppose:

```
lower = 3  
upper = 6
```

Then:

```
3 <= 4 + nums[j] <= 6
```

Subtract 4:

```
3 - 4 <= nums[j] <= 6 - 4
```

Therefore:

```
-1 <= nums[j] <= 2
```

🔥 This is the key transformation.

Instead of checking:

```
4 + nums[j]
```

for every j , we can simply ask:

How many numbers are between -1 and 2 ?

That's much easier if the array is sorted.

4. Why do we sort?

Suppose:

```
nums = [0, 1, 7, 4, 4, 5]
```

Sort it:

```
[0, 1, 4, 4, 5, 7]
```

Now suppose:

```
nums[i] = 1
```

We need:

```
3 <= 1 + nums[j] <= 6
```

Therefore:

```
2 <= nums[j] <= 5
```

In sorted array:

```
[0, 1, 4, 4, 5, 7]  
      ↑      ↑  
      2      5
```

There are three values:

```
4, 4, 5
```

So there are 3 valid partners for 1.

This is where binary search becomes useful.

5. Why `lower_bound`?

C++ provides:

```
lower_bound()
```

It finds the **first position where value $\geq$ target**.

Example:

```
nums = [0, 1, 4, 4, 5, 7]
```

Suppose:

```
target = 2
```

Then:

```
lower_bound(nums.begin(), nums.end(), 2)
```

points to:

```
4
```

because 4 is the first value satisfying:

```
value  $\geq$  2
```

So:

```
lower_bound = first valid element
```

6. Why `upper_bound`?

`upper_bound()` finds the **first position where value $>$ target**.

Example:

```
nums = [0, 1, 4, 4, 5, 7]
```

Suppose:

```
target = 5
```

Then:

```
upper_bound(..., 5)
```

points to:

```
7
```

because 7 is the first value:

```
value  $>$  5
```

So:

```
upper_bound = position AFTER the last valid element
```

Therefore:

```
lower_bound  $\rightarrow$  first allowed  
upper_bound  $\rightarrow$  first not allowed
```

So the number of valid elements is:

```
upper_bound - lower_bound
```

7. Now look at your code

Let's go line by line.

```
long long countFairPairs(  
    vector<int>& nums,  
    int lower,  
    int upper  
)
```

We receive:

```
nums  
lower  
upper
```

Step 1: Sort

```
sort(nums.begin(), nums.end());
```

Example:

Before:

```
[0, 1, 7, 4, 4, 5]
```

After:

```
[0, 1, 4, 4, 5, 7]
```

Why?

Because binary search requires the data to be sorted.

8. Why long long ans?

```
long long ans = 0;
```

The number of pairs can be very large.

For example, if:

```
n = 100000
```

number of pairs can be around:

```
100000 × 99999 / 2  
≈ 5 billion
```

That doesn't fit in a normal 32-bit `int`.

So:

```
long long
```

is necessary.

9. Loop through every element

```
for (int i = 0; i < n; i++) {
```

We are saying:

Let's fix `nums[i]` and find how many elements after it can form a fair pair.

Suppose:

```
nums = [0, 1, 4, 4, 5, 7]
```

When:

```
i = 2
```

we have:

```
nums[i] = 4
```

Now we want to find all valid partners.

10. Calculate the minimum partner

Your code:

```
long long leftValue =  
(long long)lower - nums[i];
```

Remember our condition:

```
lower <= nums[i] + nums[j]
```

Subtract `nums[i]`:

```
lower - nums[i] <= nums[j]
```

Therefore:

```
nums[j] >= lower - nums[i]
```

So:

```
leftValue = lower - nums[i];
```

is the **smallest value that can be paired with** `nums[i]`.

11. Calculate the maximum partner

Similarly:

```
long long rightValue =  
(long long)upper - nums[i];
```

Because:

```
nums[i] + nums[j] <= upper
```

Subtract `nums[i]`:

```
nums[j] <= upper - nums[i]
```

Therefore:

```
rightValue = upper - nums[i]
```

is the **largest value that can be paired with** `nums[i]`.

So now we have:

```
leftValue <= nums[j] <= rightValue
```

🔥 That's the whole mathematical transformation.

12. Why `i + 1`?

Your code:

```
nums.begin() + i + 1
```

This is extremely important.

We only want:

```
i < j
```

The current element is:

```
nums[i]
```

So possible partners are only:

```
i + 1, i + 2, ..., n - 1
```

We must NOT search:

```
0 ... i
```

Otherwise we would count:

- the same element with itself
- pairs we've already counted

For example:

```
i = 2
```

we only search:

```
3, 4, 5, ...
```

That's why:

```
nums.begin() + i + 1
```

is used.

13. Find first valid partner

```
auto left = lower_bound(
    nums.begin() + i + 1,
    nums.end(),
    leftValue
);
```

Suppose:

```
nums = [0, 1, 4, 4, 5, 7]
```

and:

```
i = 1
nums[i] = 1
lower = 3
upper = 6
```

Then:

```
leftValue = 3 - 1
           = 2
```

We need:

```
nums[j] >= 2
```

Search:

```
[4, 4, 5, 7]
```

lower_bound(2) points to:

```
4
```

So:

```
left → first valid partner
```

14. Find first invalid partner

Now:

```
auto right = upper_bound(
    nums.begin() + i + 1,
    nums.end(),
    rightValue
);
```

Here:

```
rightValue = upper - nums[i]
```

For our example:

```
rightValue = 6 - 1  
           = 5
```

We need:

```
nums[j] <= 5
```

upper_bound(5) returns the first element:

```
> 5
```

So:

```
[4, 4, 5, 7]  
      ↑  
    right
```

right points to 7.

Therefore everything between:

```
left
```

and:

```
right
```

is valid.

15. Why right - left?

This is a very important C++ concept.

Suppose:

```
[4, 4, 5, 7]  
  ↑   ↑  
left  right
```

Positions:

```
left  = 0  
right = 3
```

Number of elements:

```
3 - 0 = 3
```

Those elements are:

```
4  
4  
5
```

Exactly 3 valid partners.

Therefore:

```
ans += right - left;
```

16. Complete example

Let's execute the algorithm manually.

Given:

```
nums = [0, 1, 7, 4, 4, 5]  
lower = 3  
upper = 6
```

After sorting:

```
[0, 1, 4, 4, 5, 7]
```

i = 0

```
nums[i] = 0
```

We need:

```
3 <= 0 + nums[j] <= 6
```

Therefore:

```
3 <= nums[j] <= 6
```

Search:

```
[1, 4, 4, 5, 7]
```

First >= 3:

```
4
```

First > 6:

```
7
```

Valid:

```
4, 4, 5
```

Count:

```
3
```

i = 1

```
nums[i] = 1
```

Need:

```
3 <= 1 + nums[j] <= 6
```

Therefore:

```
2 <= nums[j] <= 5
```

Search:

```
[4, 4, 5, 7]
```

Valid:

```
4, 4, 5
```

Count:

```
3
```

Total:

```
6
```

i = 2

```
nums[i] = 4
```

Need:

```
3 <= 4 + nums[j] <= 6
```

Therefore:

```
-1 <= nums[j] <= 2
```


But remaining elements are:

[4, 5, 7]

None work.

Count:

0

i = 3

nums[i] = 4

Remaining:

[5, 7]

Need:

-1 <= nums[j] <= 2

None.

i = 4

nums[i] = 5

Remaining:

[7]

Need:

-2 <= nums[j] <= 1

None.

i = 5

No elements remain.

So:

answer = 6

17. Visualizing the binary search

This is probably the best mental picture to remember.

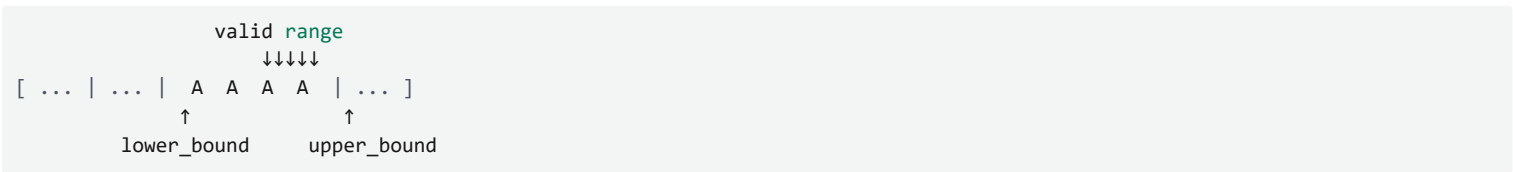
For a fixed:

nums[i]

we want:

lower - nums[i] <= nums[j] <= upper - nums[i]

In sorted array:



lower_bound gives:

first valid

upper_bound gives:

first invalid

Therefore:

```
number of valid = right - left
```

18. The algorithm in one picture

The entire algorithm is:



That's the solution.

19. Why this is much faster

Brute force:

```
For every i
  check every j
```

Complexity:

```
O(n2)
```

Our solution:

Sorting

```
sort(...)
```

takes:

```
O(n log n)
```

For every element

We perform:

```
lower_bound → O(log n)
upper_bound → O(log n)
```

for n elements:

```
O(n log n)
```

Total:

```
O(n log n) + O(n log n)
```

which is:

```
O(n log n)
```

Space:

```
O(1)
```

apart from the sorting implementation's stack/internal requirements.

20. Why cast to `long long`?

You have:

```
(long long)lower - nums[i]
```

and:

```
(long long)upper - nums[i]
```

This prevents arithmetic from being performed as `int`.

It's generally a good defensive choice here because the calculated boundary can exceed the `int` range depending on constraints.

Also, the answer itself is:

```
long long
```

because there can be up to roughly:

```
n(n-1)/2
```

pairs.

21. One subtle point: `lower_bound` and `upper_bound`

You should memorize their exact meanings:

`lower_bound(x)`

First position where value $\geq x$

Example:

```
[1, 3, 3, 5, 7]
```

```
lower_bound(3)
```

points to the first 3.

`upper_bound(x)`

First position where value $> x$

```
upper_bound(3)
```

points to:

```
5
```

So:

```
lower_bound(x) →  $\geq x$ 
```

```
upper_bound(x) →  $> x$ 
```

This distinction is **extremely important** in binary-search problems.

22. Why not use `lower_bound(rightValue)`?

Because the upper boundary is inclusive.

We need:

```
nums[j] <= rightValue
```

Suppose:

```
rightValue = 5
```

and array:

```
[4, 5, 5, 7]
```

We want both 5s.

If we used:

```
lower_bound(5)
```

we would get the **first 5**, which would exclude the 5s from our range calculation.

Instead:

```
upper_bound(5)
```

gives:

```
7
```

so both 5s are included.

This is the classic pattern:

```
[left, right]
```

can be found using:

```
lower_bound(left)
upper_bound(right)
```

23. The bigger problem-solving lesson

This problem teaches a very useful way of approaching array problems.

When you see:

Find pairs such that:

$$L \leq a[i] + a[j] \leq R$$

don't immediately think:

nested loops

Instead ask:

Question 1

Can I fix one element?

```
fix a[i]
```

Question 2

What does the condition become?

$$L \leq a[i] + a[j] \leq R$$

Rearrange:

$$L - a[i] \leq a[j] \leq R - a[i]$$

Now it is a **range query**.

Question 3

Can I make the array sorted?

Yes.

Question 4

Once sorted, can I find the range using binary search?

Yes.

Therefore:

Sort + Binary Search

24. General pattern you should remember

Whenever you see something like:

$$a[i] + a[j] \geq X$$

fix `a[i]`:

```
a[j] >= X - a[i]
```

Now use:

```
lower_bound()
```

If you see:

```
a[i] + a[j] <= X
```

fix `a[i]`:

```
a[j] <= X - a[i]
```

Now use:

```
upper_bound()
```

If you see:

```
L <= a[i] + a[j] <= R
```

fix `a[i]`:

```
L - a[i] <= a[j] <= R - a[i]
```

Then:

```
lower_bound(L - a[i])
upper_bound(R - a[i])
```

and:

```
count = right - left;
```

🔥 This is a pattern worth remembering.

25. Another way to solve the same problem

There is another beautiful approach.

Instead of directly counting:

```
L <= sum <= R
```

we can calculate:

```
count(sum <= R)
-
count(sum < L)
```

or equivalently:

```
count(sum <= R)
-
count(sum <= L - 1)
```

So:

```
fair pairs
=
pairs with sum <= upper
-
pairs with sum <= lower - 1
```

This leads to a **two-pointer** solution after sorting.

That solution is also $O(n \log n)$ because of sorting, but after sorting the counting itself is $O(n)$.

It's another important pattern:

```
count(L <= x <= R)
=
count(x <= R) - count(x < L)
```

You'll see this idea frequently in competitive programming.

26. How should you approach these problems in general?

When you encounter a new pair-counting problem, follow this checklist.

Step 1 – Understand the condition

Write exactly what needs to be true.

For this problem:

```
lower <= nums[i] + nums[j] <= upper
```

Step 2 – Try brute force first

Think:

```
for i
  for j
```

This gives you the correct logical definition.

Don't skip this step.

Step 3 – Look for what can be fixed

Ask:

If I fix `i`, can I describe all valid `j`s?

Here:

```
lower - nums[i] <= nums[j] <= upper - nums[i]
```

Yes!

Step 4 – Recognize the structure

We now need:

How many numbers lie inside a value range?

That's a **range counting** problem.

Step 5 – Ask whether sorting helps

After sorting:

```
[1, 2, 4, 4, 5, 8, 10]
```

a range becomes a contiguous section.

For example:

```
4 <= x <= 8

[1, 2, | 4, 4, 5, 8 | 10]
  ↑      ↑
  4      8
```

That's perfect for binary search.

Step 6 – Decide which binary search

For:

```
x >= L
```

use:

```
lower_bound(L)
```

For:

```
x <= R
```

use:

```
upper_bound(R)
```

Therefore:

```
[L, R]
```

becomes:

```
upper_bound(R) - lower_bound(L)
```

27. The most important mental transformation

I would remember this problem as:

```
PAIR PROBLEM
↓
Fix one element
↓
Turn pair condition into condition on other element
↓
lower <= a[i] + a[j] <= upper
      ↓
lower-a[i] <= a[j] <= upper-a[i]
      ↓
    RANGE QUERY
      ↓
    SORT ARRAY
      ↓
    BINARY SEARCH
```

That's the actual intuition.

Don't memorize the final code:

```
lower_bound(...)
upper_bound(...)
```

Memorize the **reasoning chain** that produces those lines.

28. Your code, conceptually simplified

Your entire code can be understood as this pseudocode:

```
sort(nums)

answer = 0

for each i:

    minimum_allowed =
        lower - nums[i]

    maximum_allowed =
        upper - nums[i]

    search only elements after i

    left =
        first element >= minimum_allowed

    right =
        first element > maximum_allowed

    valid_elements =
        right - left

    answer += valid_elements

return answer
```

Once you understand this pseudocode, the C++ implementation becomes straightforward.

One final thing to practice

After understanding this problem, try solving these three variations **without looking at the solution**:

1. **Count** pairs whose sum $\geq X$
2. **Count** pairs whose sum $\leq X$
3. **Count** pairs whose sum is between L and R

If you can derive the binary-search boundaries yourself for all three, you've actually learned the pattern rather than just this particular LeetCode problem.